

클라우드 인프라 교육 · 2일차 오후

도커 환경 실습

퍼블릭 클라우드(EC2) 환경에서 Docker 실습

실습 1 Docker 설치

실습 2 컨테이너 기본 명령

실습 3 Dockerfile 빌드 · 3-B commit

실습 4 ECR에 Push / Pull

실습 5 Docker Compose

실습 6 볼륨·네트워크 · 종합 정리

오늘 오후의 목표

- **생명주기 체득** — pull → run → stop → rm의 컨테이너 생명주기를 손으로 익힌다.
- **나만의 이미지** — Dockerfile을 작성해 이미지를 빌드하고 실행한다.
- **배포 흐름 완주** — build → push(ECR) → pull → run 표준 배포 흐름을 끝까지 수행한다.
- **멀티 컨테이너** — Compose로 웹+DB 조합을 한 번에 띄운다.

어제 개념과 오늘 실습의 대응

어제 오후에 배운 개념이 오늘 전부 명령어로 재현된다

- 이미지·컨테이너·Dockerfile·레지스트리
- 배포 표준 흐름(build→push→pull→run)
- Compose의 선언적 관리 / 볼륨·네트워크

[Day1 복습] 컨테이너가 왜 필요했나

- **VM의 아쉬움** — OS를 통째로 담아 수 GB·부팅 수 분. 앱 하나 옮기기엔 너무 무겁다
- **컨테이너의 답** — 호스트 OS 커널을 공유하고 앱+실행환경만 격리 → 수십 MB·1초 기동
- **구조 그림 기억** — VM: 앱/게스트OS×N/하이퍼바이저 vs 컨테이너: 앱×N/Docker/호스트OS 하나
- **오늘의 구성** — "VM 위에 컨테이너" 표준 조합: 오전에 만든 EC2(VM) 위에 지금 Docker를 얹는다

실습 1

Docker 설치

EC2 재시작 · 설치 · 권한 설정

준비 — 오전 인스턴스 재가동

1

EC2 콘솔에서 중지해 둔 my-first-server 선택 → [인스턴스 시작]

2

상태 running + 상태 검사 2/2 확인 (1~2분 소요)

3

퍼블릭 IP가 오전과 달라졌을 수 있음 — 새 IP 확인

4

[연결] → Instance Connect로 터미널 접속 or PuTTY로 접속

Docker 설치 명령

```
sudo dnf install -y docker      # docker 패키지 설치
                                # sudo: 관리자 권한, -y: 확인 자동 yes
sudo systemctl start docker     # docker 서비스(데몬) 시작
sudo systemctl enable docker    # 재부팅 시 자동 시작 등록
sudo systemctl status docker    # 상태 확인 - active (running) 이면 정상
```

systemctl = 리눅스 서비스 관리 명령. 어제 배운 Docker 데몬(dockerd)을 시작하는 것

권한 설정 — sudo 없이 docker 쓰기

```
sudo usermod -aG docker ec2-user
# ec2-user를 docker 그룹에 추가
# -aG : 기존 그룹 유지(a)하며 그룹 추가(G)

newgrp docker      # 즉시 권한 적용
# → 다시 [연결]로 접속한 뒤 확인:
docker ps          # sudo 없이 동작하면 성공
```

그룹 변경은 재로그인해야 적용된다 — exit 후 재접속을 잊지 말 것

설치 확인

```
docker --version
# Docker version 2x.x.x ... 출력되면 설치 성공

docker ps
# CONTAINER ID   IMAGE    ... (빈 표가 정상 - 아직 컨테이너 없음)

docker info | head -20
# 데몬 상세 정보 (스토리지 드라이버, 컨테이너 수 등)
```

빈 표 = 오류가 아니라 "실행 중인 컨테이너가 없다"는 정상 응답

실습 1 체크포인트

- ☐ `docker --version`이 버전을 출력하는가?
- ☐ `sudo` 없이 `docker ps`가 동작하는가?
- ☐ `systemctl status docker`가 active인가?

실습 2

컨테이너 기본 명령

nginx로 배우는 pull · run · ps · logs · stop · rm

[Day1 복습] 이미지 · 컨테이너 · 레지스트리

- **이미지** — 앱+실행환경을 포장한 읽기 전용 설계도
- **컨테이너** — 그 설계도를 실행한 실체(프로세스). 이미지 1개 → 컨테이너 N개
- **레지스트리** — 이미지를 push/pull하는 창고 (Docker Hub=공개, ECR=우리 전용)
- **지금부터** — pull(창고에서 받기) → run(실행) → ps/logs(관찰) → stop/rm(정리)을 손으로 돌린다

① 이미지 내려받기 — pull

```
docker pull nginx
# Docker Hub에서 nginx 공식 이미지 다운로드
# "Downloading" 줄이 여러 개 = 어제 배운 레이어(Layer)들

docker images
# REPOSITORY    TAG        SIZE
# nginx          latest     ~190MB
# 내려받은 이미지 목록 확인
```

레이어 단위 다운로드를 눈으로 확인 — 같은 레이어는 다음부터 재사용된다

② 컨테이너 실행 — run

```
docker run -d -p 80:80 --name my-web nginx
```

```
# run      : 이미지를 컨테이너로 실행
# -d      : 백그라운드 실행 (터미널을 계속 사용 가능)
# -p 80:80 : 서버(호스트) 80 → 컨테이너 80 포트 연결
# --name   : 컨테이너 이름 지정 (없으면 임의 이름)
# nginx    : 사용할 이미지

# 출력되는 긴 문자열 = 컨테이너 ID
```

1초 만에 웹 서버 1대가 떴다 — 설치·설정 과정 없이

③ 상태 확인 — ps

```
docker ps
```

# CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
# a1b2c3...	nginx	Up 10 seconds	0.0.0.0:80->80/tcp	my-web

STATUS Up = 실행 중

PORTS 0.0.0.0:80->80 = 모든 IP의 80 요청을 컨테이너 80으로

ps는 앞으로 가장 많이 치게 될 명령 — 항상 상태부터 확인

④ 브라우저로 확인하기

1

보안그룹에 HTTP(80) — 0.0.0.0/0 인바운드 규칙 추가 (웹 공개용이므로 전체 개방)

2

내 PC 브라우저에서 `http://EC2퍼블릭IP` 접속

3

"Welcome to nginx!" 페이지가 보이면 성공

4

(안 보이면) ① `docker ps`로 Up 확인 ② 보안그룹 80 확인 ③ http인지 확인(https 아님)

방금 일어난 일의 의미

**웹 서버를 설치·설정한 것이 아니라
완성품 이미지를 받아 1초 만에 실행했다**

- 이것이 컨테이너의 힘 — 실행 환경째 유통
- 같은 명령이면 어느 서버에서든 같은 결과 (환경 일관성)

⑤ 로그 확인 — logs

```
docker logs my-web
# 컨테이너의 표준 출력(접속 기록 등) 확인
# 방금 브라우저 접속 기록(GET / HTTP/1.1)이 보인다

docker logs -f my-web
# -f : 실시간으로 이어 보기 (tail -f 와 같음)
# 브라우저를 새로고침하면 로그가 실시간 추가
# Ctrl + C 로 빠져나오기
```

컨테이너 장애 분석의 첫 명령이 logs — 습관화

⑥ 컨테이너 내부 들어가기 — exec

```
docker exec -it my-web bash
# exec : 실행 중 컨테이너에서 명령 실행
# -it  : 대화형 터미널로
# bash : 셸 실행 → 컨테이너 내부로 진입

ls /usr/share/nginx/html    # nginx 웹 문서 폴더 확인
cat /etc/os-release         # 컨테이너 내부 OS 확인
exit                        # 컨테이너에서 빠져나오기
```

호스트(Amazon Linux)와 컨테이너 내부(Debian 계열)의 OS가 다를음을 확인 — 격리의 실감

⑦ 중지·시작·재시작

```
docker stop my-web      # 컨테이너 중지 (프로세스 종료)
docker ps               # 목록에서 사라짐
docker ps -a            # -a: 중지된 것 포함 전체 - Exited 상태로 존재

docker start my-web     # 다시 시작
docker restart my-web   # 재시작 (설정 반영 등에 사용)
```

stop해도 컨테이너는 삭제되지 않는다 — ps와 ps -a의 차이를 눈으로 확인

⑧ 삭제 — rm / rmi

```
docker rm my-web      # 오류! 실행 중에는 삭제 불가
docker rm -f my-web   # -f : 강제 종료 후 삭제
docker ps -a          # 목록에서 완전히 사라짐

docker rmi nginx       # 이미지 삭제 (컨테이너가 없을 때 가능)
docker images          # 확인
```

컨테이너(실행체)와 이미지(원본)는 별개로 삭제 — 관계를 다시 확인

[예제] 다른 이미지도 같은 방식 ① — Apache 웹서버

```
docker run -d -p 8080:80 --name my-httpd httpd
# nginx 대신 Apache(httpd) - 호스트 8080 포트 사용
# (80은 my-web nginx가 쓰는 중이므로)

# 보안그룹에 8080 추가 후 브라우저: http://퍼블릭IP:8080
# "It works!" 페이지 = Apache 기본 화면

docker ps    # nginx(80)와 httpd(8080)가 나란히 실행 중
```

포트만 나누면 한 서버에 웹서버 여러 종을 동시 운영 — 컨테이너 집적의 실감

[예제] 다른 이미지도 같은 방식 ② — MySQL DB

```
docker run -d --name my-db \  
  -e MYSQL_ROOT_PASSWORD=edu1234 \  
  -e MYSQL_DATABASE=testdb \  
  mysql:8  
# -e : 환경변수로 초기 설정 주입 (이미지 문서에 정의된 변수들)  
  
docker logs my-db | tail -5      # "ready for connections" 확인  
docker exec -it my-db mysql -uroot -pedu1234 -e "SHOW DATABASES;"  
# 컨테이너 안의 mysql 클라이언트로 testdb 생성 확인
```

DB 서버 설치가 명령 한 줄 — 단, 데이터 보존은 실습 6의 볼륨과 결합해야 완성

[예제] 다른 이미지도 같은 방식 ③ — Redis

```
docker run -d --name my-redis redis

docker exec -it my-redis redis-cli
127.0.0.1:6379> SET greeting "hello docker"
127.0.0.1:6379> GET greeting
"hello docker"
127.0.0.1:6379> exit

# 예제 정리 (다음 실습을 위해)
docker rm -f my-httpd my-db my-redis
```

웹서버·DB·캐시 전부 같은 패턴(run -d --name) — "포장 방식이 하나"라는 표준화의 힘

실습 2 명령 총정리

명령	역할	비고
docker pull	이미지 다운로드	레이어 단위
docker run -d -p	컨테이너 백그라운드 실행+포트 연결	없는 이미지는 자동 pull
docker ps / ps -a	실행 중 / 전체 목록	상태 확인 1순위
docker logs (-f)	로그 (실시간) 확인	장애 분석 1순위
docker exec -it ~ bash	내부 진입	설정·파일 확인
docker stop / start	중지 / 시작	중지≠삭제
docker rm (-f) / rmi	컨테이너 / 이미지 삭제	실행 중은 -f

실습 3

Dockerfile로 나만의 이미지 만들기

작성 → build → run → 수정 → 재빌드

Dockerfile과 레이어

- **Dockerfile** — 이미지를 만드는 절차를 적은 텍스트 레시피. "이 OS에서 시작해, 이걸 복사하고, 이렇게 실행하라"
- **파일이 곧 문서** — 레시피가 남으니 누구든 같은 이미지를 재현
- **레이어** — 지시어 한 줄 = 층 하나. 바뀐 층만 다시 만들어 재빌드가 빠르다 (오늘 CACHED로 직접 목격)

① 작업 폴더와 웹페이지 준비

```
mkdir myapp && cd myapp      # 작업 폴더 생성 후 이동

vi index.html                # 웹페이지 작성 (vim 편집기)

# --- 편집기에 입력 ---
<h1>Hello, My First Docker Image!</h1>
<p>만든 사람: 본인 이름</p>
# --- Ctrl+O, Enter 저장 → Ctrl+X 종료 ---

cat index.html              # 내용 확인
```

저장은 :wq, 종료는 :q!

② Dockerfile 작성

```
nano Dockerfile          # 파일명은 정확히 Dockerfile (대문자 D)

# --- 편집기에 입력 ---
FROM nginx
COPY index.html /usr/share/nginx/html/
# --- 저장 후 종료 ---

# FROM : nginx 이미지를 출발점으로
# COPY : 내 index.html을 이미지 안 웹 폴더에 복사
```

③ 이미지 빌드 — build

```
docker build -t my-web-image .
```

```
# build : Dockerfile로 이미지 생성  
# -t    : 이미지 이름(태그) 지정  
# .     : 빌드 컨텍스트 = 현재 폴더 (Dockerfile 위치)
```

```
# Step 1/2 : FROM nginx  
# Step 2/2 : COPY index.html ...  
# Successfully tagged my-web-image:latest
```

```
docker images      # my-web-image 확인
```

④ 내 이미지 실행과 확인

```
docker run -d -p 80:80 --name my-web2 my-web-image
```

브라우저에서 <http://EC2퍼블릭IP> 새로고침

→ "Hello, My First Docker Image!" 표시되면 성공

404/기본페이지가 나오면:

```
docker ps                                # my-web2가 Up인지
```

```
docker logs my-web2                      # 오류 로그 확인
```

```
docker exec -it my-web2 ls /usr/share/nginx/html
```

남의 이미지 실행(실습2)에서 → 내 이미지 실행으로 한 단계 상승

⑤ 수정 → 재빌드 사이클

```
nano index.html          # 내용 수정 (예: 버전 2 문구 추가)

docker build -t my-web-image:v2 .    # v2 태그로 재빌드
# Step 1/2 CACHED ← FROM 레이어는 캐시 재사용!
# Step 2/2 변경된 COPY만 다시 실행 → 빌드가 빠른 이유

docker rm -f my-web2
docker run -d -p 80:80 --name my-web2 my-web-image:v2
# 브라우저 새로고침 → 수정 내용 확인
```

수정→빌드→교체가 컨테이너식 배포의 기본 사이클. 태그(v2)로 버전을 구분하는 습관

왜 컨테이너를 "고치지 않고 교체"하는가

**실행 중인 컨테이너를 수정하지 않고
새 이미지를 빌드해 통째로 교체했다**

- 컨테이너 내부를 수동으로 고치면 이미지와 실체가 어긋남(구성 편차)

실습 3-B

`docker commit` — 컨테이너를 이미지로

또 하나의 이미지 제작법, 그리고 Dockerfile이 정석인 이유

commit이란

**실행 중인 컨테이너의 현재 상태를 통째로 찍어서
새 이미지로 저장하는 명령**

- Dockerfile이 "명세서로 만들기"라면, commit은 "완성된 컨테이너를 그대로 이미지화하기"
- VM의 스냅샷과 같은 발상 — 지금 상태를 보존

commit 실습 ① — 컨테이너 안에서 수정하기

```
docker run -d -p 80:80 --name web-edit nginx

docker exec -it web-edit bash      # 컨테이너 내부 진입
echo "<h1>Edited inside container!</h1>" \
  > /usr/share/nginx/html/index.html
exit

# 브라우저 새로고침 → 수정된 페이지 확인
# 이 변경은 지금 "이 컨테이너 안"에만 존재한다
# → 컨테이너를 지우면 사라질 운명
```

실습 2에서 배운 exec 응용 — 지금은 일부러 컨테이너를 직접 수정해 본다

commit 실습 ② — 상태를 이미지로 저장

```
docker commit web-edit my-web-commit:v1
# web-edit 컨테이너의 현재 상태 → my-web-commit:v1 이미지

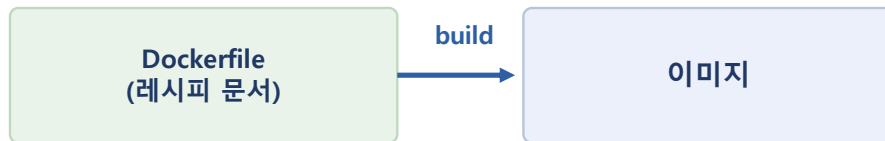
docker images
# my-web-commit v1 ... (새 이미지 등장)

# 검증: 원본 컨테이너를 지우고 commit 이미지로 재실행
docker rm -f web-edit
docker run -d -p 80:80 --name web-restored my-web-commit:v1
# 브라우저 → "Edited inside container!" 가 그대로!
# 수정 내용이 이미지에 보존되었다
```

commit한 이미지도 tag를 붙여 ECR에 push 가능 — 이미지가 된 이상 유통 방식은 동일

두 가지 이미지 제작법 비교

Dockerfile 방식 (실습 3)



과정이 문서로 남음 · 누구나 재현 · 변경 이력 관리

commit 방식 (실습 3-B)



빠르지만 "무엇을 어떻게 바꿨는지" 기록이 없음

실무 원칙: 이미지는 Dockerfile로 만든다 (재현성·이력·협업)
commit의 용도: 장애 상황 보존(디버깅용 스냅샷), 급한 임시 조치, 실험 상태 백업

commit을 쓰면 안 되는 이유를 직접 체험하기

- **질문** — 방금 만든 my-web-commit:v1, 3개월 뒤의 내가 "무엇이 바뀐 이미지"인지 알 수 있을까?
- **확인** — `docker history my-web-commit:v1` → 마지막 레이어가 그냥 "bash"로만 기록됨. 무슨 작업인지 알 수 없다
- **반면 Dockerfile 이미지** — history에 COPY index.html... 이 그대로 남아 과정 추적 가능
- **결론** — commit은 "할 줄 알아야 하지만, 평소엔 안 쓰는" 명령. 정석은 Dockerfile → build → push

실습 3-B 정리

```
docker rm -f web-restored
docker rmi my-web-commit:v1
docker ps -a && docker images # 실습 3의 my-web-image만 남긴다
```

다음 실습(ECR)에서는 Dockerfile로 만든 my-web-image를 push한다

실습 4

ECR에 Push / Pull

배포 표준 흐름의 완성 — 레지스트리 실습

[Day1 복습] 배포의 표준 흐름과 레지스트리

- **유통 그림 기억** — 개발 PC(build) → push → 레지스트리 → pull → 운영 서버들(run)
- **왜 창고를 거치나** — "한 번 만들어 어디서나 실행"은 이미지를 공유할 창고가 있어야 성립
- **ECR** — AWS의 프라이빗 레지스트리. 접근 통제는 IAM(오전 복습의 그 IAM)으로
- **서버의 권한** — EC2가 ECR에 접근하는 열쇠는 액세스 키가 아니라 IAM 역할(Role) — 오전에 배운 원칙 그대로

왜 레지스트리에 올리는가

지금 my-web-image는 이 EC2 안에만 존재한다
→ 레지스트리에 올려야 다른 서버·다른 사람이 쓸 수 있다

- build(내 서버) → push(창고) → pull(운영 서버) → run — 배포의 표준 흐름
- ECR = AWS의 프라이빗 이미지 창고. IAM으로 접근 통제

① ECR 리포지토리 생성 (콘솔)

1

AWS 콘솔 검색창에 ECR 입력 → [Amazon ECR] 이동

2

[리포지토리 생성] 클릭 — 프라이빗 유지

3

이름: my-web-image-이니셜 (예: my-web-image-hgd)

4

[생성] → 목록에서 리포지토리 URI 확인 (계정번호.dkr.ecr.ap-northeast-2.amazonaws.com/...)

③ ECR 로그인

```
aws ecr get-login-password --region ap-northeast-2 \  
| docker login --username AWS \  
--password-stdin 계정번호.dkr.ecr.ap-northeast-2.amazonaws.com
```

1) aws ecr get-login-password : 임시 비밀번호 발급

2) | (파이프) : 그 출력을 다음 명령의 입력으로 전달

3) docker login : ECR에 로그인

"Login Succeeded" 가 뜨면 성공

계정번호는 ECR 콘솔의 URI에서 복사

ECR 콘솔의 [푸시 명령 보기] 버튼에 이 명령이 그대로 있다 — 복사해서 사용해도 됨

④ 태그 붙이기 — 배송지 라벨

```
docker tag my-web-image:latest \  
계정번호.dkr.ecr.ap-northeast-2.amazonaws.com/my-web-image-hgd:latest
```

```
# tag = 이미지에 "어느 창고의 어느 칸으로 갈지" 주소를 붙이는 것  
# 원본이 복사되는 게 아니라 같은 이미지에 이름이 하나 더 생김
```

```
docker images  
# my-web-image 와 긴 ECR 주소 이미지의 IMAGE ID가 동일함을 확인
```

IMAGE ID가 같다 = 같은 이미지에 라벨만 2개라는 뜻

⑤ 업로드 — push

```
docker push 계정번호.dkr.ecr.ap-northeast-2.amazonaws.com/my-web-image-hgd:latest

# 레이어 단위로 업로드 진행률 표시
# nginx 공통 레이어가 크고, 내 index.html 레이어는 작음

# 확인: ECR 콘솔 → 리포지토리 → 이미지 목록에
#       latest 태그 이미지가 보이면 성공
#       [스캔] 기능으로 취약점 점검도 가능 (어제 배운 이미지 보안)
```

push 완료 = 내 이미지가 이 서버를 떠나 조직의 자산이 된 순간

⑥ Pull 검증 — 배포 흐름의 반대편

```
# "다른 서버"를 가정하고 로컬 이미지를 지운 뒤 다시 받아본다

docker rm -f my-web2                # 컨테이너 정리
docker rmi my-web-image my-web-image:v2  # 로컬 이미지 삭제
docker rmi 계정번호.dkr...../my-web-image-hgd:latest
docker images                       # 비어 있음 확인

docker pull 계정번호.dkr...../my-web-image-hgd:latest
docker run -d -p 80:80 --name from-ecr 계정번호.dkr...../my-web-image-hgd:latest
# 브라우저 확인 → 내 페이지가 그대로! 배포 흐름 완주
```

방금 한 것이 실제 운영 배포와 동일한 원리 — 운영 서버는 pull과 run만 한다

실습 5

Docker Compose

웹 + DB 멀티 컨테이너를 선언으로 관리

[Day1 복습] 선언적 관리

- **명령형 vs 선언형** — "이거 해라"가 아니라 "이 상태여야 한다"를 파일로 선언
- **어제의 연결 고리** — Compose(한 서버의 선언) → K8s(클러스터의 선언+자가치유)
- **오늘 체감할 것** — yml 한 파일 선언 → up 한 번에 웹+DB+네트워크가 통째로 생기는 순간
- **MSA 미리보기** — 웹과 DB를 분리해 조합하는 이 구성이 어제 배운 서비스 분리 사고의 축소판

① Compose 설치와 확인

```
sudo dnf install -y docker-compose-plugin
# compose 플러그인 설치 (docker compose 명령 제공)

docker compose version
# Docker Compose version v2.x.x 출력되면 성공

mkdir ~/compose-lab && cd ~/compose-lab
# 실습 폴더 생성.이동
```

구버전 docker-compose(하이픈)와 신버전 docker compose(공백) — 오늘은 신버전 사용

② docker-compose.yml 작성

```
vi docker-compose.yml

# --- 편집기에 입력 (들여쓰기는 스페이스 2칸!) ---
services:
  web:
    image: nginx
    ports:
      - "80:80"
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: edu1234
# --- 저장 후 종료 ---
```

yaml은 들여쓰기가 문법이다 — 탭 금지, 스페이스 2칸 통일.

③ 한 번에 실행 — up

```
docker compose up -d
# yml에 선언된 모든 서비스(web, db)를 생성·실행
# -d : 백그라운드
# 네트워크도 자동 생성됨 (compose-lab_default)

docker compose ps
# NAME                STATUS
# compose-lab-web-1   Up
# compose-lab-db-1    Up

# 브라우저: http://EC2퍼블릭IP → nginx 페이지
```

명령 한 번에 두 컨테이너 + 전용 네트워크 — 선언적 관리의 체감

④ 서비스 간 통신 확인

```
docker compose logs db | head -20
# MySQL 초기화 로그 - ready for connections 확인

docker exec -it compose-lab-web-1 bash
apt update && apt install -y iputils-ping # (확인용 도구 설치)
ping -c 2 db
# 컨테이너 이름 "db"로 통신이 된다!
exit
```

같은 Compose 네트워크에서는 서비스 이름이 곧 주소 — 어제 배운 컨테이너 네트워크의 실증

Compose에서 K8s로 이어지는 길

**"무엇이 떠 있어야 하는가"를 파일로 선언했다 —
이 선언이 여러 서버로 커지면 그것이 Kubernetes다**

- Compose: 한 서버 안의 선언적 관리
- K8s: 여러 서버(클러스터)의 선언적 관리 + 자가 치유·오토스케일
- 어제 본 Deployment yaml과 오늘 yml의 철학이 같다

⑤ 정리 — down

```
docker compose down
# yml로 만든 컨테이너·네트워크를 한 번에 제거
# Removing compose-lab-web-1 ... done
# Removing compose-lab-db-1 ... done
```

```
docker compose ps    # 빈 목록
docker ps -a         # 전체 확인
```

up으로 만든 것은 down으로 — 만든 방식 그대로 정리하는 것이 원칙

실습 6

볼륨 · 네트워크 · 종합 정리

데이터 보존 실험 · 전체 리소스 정리

[Day1 복습] 컨테이너는 일회용, 데이터는 밖에

- **어제의 원칙** — 컨테이너를 지우면 내부 변경도 사라진다 → 데이터는 컨테이너 밖에 보관
- **볼륨** — 그 "밖"의 표준. 컨테이너가 죽고 새로 떠도 데이터는 유지
- **불변 인프라와 연결** — 컨테이너를 고치지 않고 교체할 수 있는 것도 데이터가 밖에 있기에 가능
- **지금부터** — 볼륨 없이/있이 두 번 실험해 이 원칙을 몸으로 확인한다

① 볼륨 실험 — 데이터는 어디에 남는가

```
# 볼륨 없이: 컨테이너 삭제 = 데이터 소멸을 확인
docker run -d --name t1 nginx
docker exec t1 sh -c "echo data > /tmp/test.txt"
docker rm -f t1
docker run -d --name t1 nginx
docker exec t1 cat /tmp/test.txt      # No such file - 사라짐!
docker rm -f t1

# 볼륨 사용: 데이터가 컨테이너 밖에 보존됨
docker run -d --name t2 -v mydata:/data nginx
docker exec t2 sh -c "echo keep > /data/test.txt"
docker rm -f t2
docker run -d --name t2 -v mydata:/data nginx
docker exec t2 cat /data/test.txt    # keep - 살아 있다!
```

-v 볼륨명:컨테이너경로. "컨테이너는 일회용, 데이터는 볼륨에"의 실증

② 볼륨·네트워크 관리 명령

```
docker volume ls          # 볼륨 목록 (mydata 확인)
docker volume inspect mydata # 실제 저장 위치 확인
docker network ls         # 네트워크 목록
    # bridge(기본), compose-lab_default(있다면) 등

docker rm -f t2
docker volume rm mydata    # 볼륨 삭제 (컨테이너 정리 후)
```

볼륨은 컨테이너를 지워도 남는다 = 지우는 것도 별도 명령

③ [정리하기] 오후 전체 리소스 정리

```
docker rm -f $(docker ps -aq)      # 모든 컨테이너 강제 삭제
# $(docker ps -aq) : 전체 컨테이너 ID 목록을 인자로 전달

docker system prune -a -f          # 미사용 이미지·네트워크 일괄 정리
docker ps -a && docker images      # 모두 비었는지 확인

# 콘솔에서:
# 1) 보안그룹 HTTP(80) 규칙 삭제
# 2) EC2 인스턴스 [중지] (내일 Terraform 비교용으로 유지)
# 3) ECR 이미지는 유지 (3일차 마지막에 리포지토리째 삭제)
```

"실행 중 컨테이너 0개, 실행 중 인스턴스 0대" — 옆 사람과 상호 확인

[미니 챌린지] 스스로 해보기 — 기본

답을 보지 않고 명령을 조합해 보세요 (10분)

- **과제 1** — httpd 이미지를 8081 포트로 실행하고, 브라우저로 확인 후 삭제하기
- **과제 2** — 실행 중인 nginx 컨테이너의 로그를 실시간으로 보면서, 브라우저 새로고침이 찍히는지 확인하기
- **과제 3** — my-web-image의 레이어 구성을 history 명령으로 확인하고, COPY 레이어를 찾기
- **힌트** — `run -d -p / logs -f / history`

[미니 챌린지] 스스로 해보기 — 응용

오늘 배운 흐름 전체를 조합하는 과제 (15분)

- **과제 4** — index.html을 "Challenge Complete - 이름"으로 수정 → v3 태그로 빌드 → 기존 컨테이너를 v3로 교체
- **과제 5** — v3 이미지를 ECR에 새 태그로 push하고, ECR 콘솔에서 태그 2개(latest, v3)를 확인하기
- **과제 6 (도전)** — compose.yml에 redis 서비스를 추가해 3개 컨테이너(web·db·redis)를 한 번에 띄우기

오늘 배운 명령 전체 지도

영역	명령	핵심
설치	<code>dnf install docker / systemctl start-enable</code>	데몬 기동
이미지	<code>pull · images · rmi · build -t · tag</code>	설계도 관리
컨테이너	<code>run -d -p · ps(-a) · logs(-f) · exec -it · stop · rm(-f)</code>	실행체 생명주기
레지스트리	<code>ecr get-login-password · login · push · pull</code>	배포 유통
Compose	<code>compose up -d · ps · logs · down</code>	선언적 멀티 컨테이너
데이터	<code>run -v · volume ls/rm</code>	보존은 볼륨에
정리	<code>rm -f \$(ps -aq) · system prune</code>	비용·위생 관리

[보강] docker run 자주 쓰는 옵션

옵션	의미	예시
-d	백그라운드 실행	run -d nginx
-p 호스트:컨테이너	포트 연결	-p 8080:80
--name	이름 지정	--name my-web
-e KEY=VALUE	환경변수 주입	-e MYSQL_ROOT_PASSWORD=xx
-v 볼륨:경로	데이터 보존	-v mydata:/data
--rm	종료 시 자동 삭제	일회성 테스트에 유용
-it	대화형 터미널	run -it ubuntu bash

[보강] 컨테이너 여러 개 동시 실행

```
# 같은 이미지로 다른 포트에 3대 실행
docker run -d -p 8081:80 --name web1 nginx
docker run -d -p 8082:80 --name web2 nginx
docker run -d -p 8083:80 --name web3 nginx
docker ps          # 3개가 각각 다른 호스트 포트로 Up

# 호스트 포트만 다르면 같은 서버에 수십 개도 가능
# - 컨테이너 밀도(집적)의 체감
docker rm -f web1 web2 web3
```

같은 80이라도 컨테이너마다 격리되어 있어 충돌하지 않는다 — 호스트 포트만 나눠 쓰면 됨

[트러블슈팅] 포트 충돌 오류

```
docker run -d -p 80:80 --name webA nginx
docker run -d -p 80:80 --name webB nginx
# Error: ... port is already allocated

# 원인: 호스트의 80 포트는 하나 – 이미 webA가 사용 중
# 해결 1) 다른 호스트 포트 사용: -p 8080:80
# 해결 2) 기존 컨테이너 정리 후 실행: docker rm -f webA

docker rm -f webA
```

가장 흔한 초보 오류 — 메시지에서 "already allocated"를 보면 포트 충돌

[보강] 상태 진단 — stats · inspect · cp

```
docker run -d -p 80:80 --name my-web nginx
```

```
docker stats --no-stream      # CPU·메모리 사용량 스냅샷  
# 컨테이너별 자원 소비 확인 (실시간은 --no-stream 제거)
```

```
docker inspect my-web | head -30  
# 상세 설정 JSON - IP, 마운트, 포트 매핑 전부
```

```
docker cp my-web:/usr/share/nginx/html/index.html ./backup.html  
# 컨테이너 ↔ 호스트 간 파일 복사
```

stats(자원)·logs(출력)·inspect(설정) — 컨테이너 진단 3종 세트

[보강] RUN 지시어 — 이미지에 도구 넣기

```
vi Dockerfile      # 기존 파일 수정

# --- 수정 내용 ---
FROM nginx
RUN apt-get update && apt-get install -y curl
    # RUN: 빌드 시점에 명령 실행 → 결과가 레이어로 저장
COPY index.html /usr/share/nginx/html/
# --- 저장 ---

docker build -t my-web-image:v3 .
    # RUN 단계에서 패키지 설치 로그가 흐른다
docker run --rm my-web-image:v3 curl --version
    # 이미지 안에 curl이 포함되었는지 확인
```

FROM(출발)·RUN(설치)·COPY(복사) 3종이면 대부분의 이미지를 만들 수 있다

[보강] 이미지 크기와 경량화 개념

- **크기 확인** — docker images의 SIZE 열. RUN으로 설치할수록 커진다
- **왜 작아야 하나** — push/pull 시간 단축, 저장 비용 절감, 공격 면적 축소(보안)
- **경량 베이스** — nginx:alpine처럼 alpine 태그는 수십 MB 수준의 초경량 리눅스 기반
- **멀티스테이지 빌드(소개)** — 빌드 도구는 앞 단계에서만 쓰고, 최종 이미지에는 결과물만 담는 기법. 이름만 알아 두기

[트러블슈팅] 빌드가 실패할 때

```
# 사례 1: "no such file or directory" (COPY 실패)
# → index.html이 현재 폴더에 있는가? ls로 확인
# → build 마지막 인자 . (빌드 컨텍스트)를 빠뜨리지 않았는가?

# 사례 2: Dockerfile을 못 찾음
# → 파일명이 정확히 Dockerfile인가? (dockerfile, Dockerfile.txt 불가)
# → pwd로 현재 위치가 myapp인지 확인

# 사례 3: RUN 단계 오류
# → 출력된 오류 메시지의 마지막 줄부터 읽는다
# → 네트워크 문제라면 잠시 후 재시도
```

빌드 오류의 대부분은 "파일 위치"와 "파일 이름" — 침착하게 ls·pwd부터

[복습] Dockerfile 지시어 요약

지시어	의미	오늘 사용
FROM	베이스 이미지 (출발점)	실습 3 ①
COPY	파일을 이미지 안으로	실습 3 ②
RUN	빌드 중 명령 실행(설치)	보강 실습
ENV	환경변수 기본값	개념
EXPOSE	사용 포트 문서화	개념
CMD	컨테이너 시작 명령	nginx 이미지에 내장

[보강] ECR 운영 기능 둘러보기

- **이미지 스캔** — 리포지토리 설정에서 [푸시 시 스캔] 활성화 → 취약점(CVE) 자동 점검. 결과는 이미지 상세에서 확인
- **수명주기 정책** — "태그 없는 이미지 7일 후 삭제" 같은 규칙으로 창고 자동 청소 → 저장 비용 관리
- **태그 불변성** — 같은 태그 덮어쓰기를 금지하는 옵션. "v1.0은 영원히 같은 이미지" 보장
- **권한 정책** — 리포지토리 단위로 누가 push/pull 가능한지 IAM으로 통제

[보강] compose 명령 지도

명령	의미
<code>docker compose up -d</code>	yml의 전체 서비스 생성·시작
<code>docker compose ps</code>	서비스 상태 목록
<code>docker compose logs (-f) 서비스명</code>	특정 서비스 로그
<code>docker compose restart 서비스명</code>	특정 서비스만 재시작
<code>docker compose down</code>	전체 제거 (컨테이너+네트워크)
<code>docker compose down -v</code>	볼륨까지 함께 제거 (데이터 삭제 주의!)

[보강] 이미지 상세 들여다보기 — history

```
docker pull nginx
docker history nginx
# 이미지가 어떤 레이어(명령)들로 만들어졌는지 이력 확인
# CREATED BY 열 = 그 레이어를 만든 Dockerfile 지시어

docker history my-web-image:v3
# 내 이미지: nginx 레이어들 + RUN(curl 설치) + COPY(index.html)
# 어제 배운 "레이어 적층 구조"를 명령으로 직접 확인
```

남의 이미지도 history로 구성 방식을 배울 수 있다

배포 흐름 최종 복습 — 오늘 완주한 길

1

Dockerfile 작성 — 이미지 레시피 (실습 3)

2

docker build — 이미지 생성, 레이어·캐시 확인 (실습 3)

3

docker push — ECR 업로드, IAM 역할로 인증 (실습 4)

4

docker pull — 로컬 삭제 후 창고에서 복원 (실습 4)

5

docker run — 어디서든 동일 실행 + Compose로 조합 (실습 2·5)

6

이 다섯 단계가 컨테이너 시대 배포의 전부다